

Kod Sağlamaştırma: Java ve Yorumlanan Diller

CEN429 Güvenli Programlama — Hafta 5

Dr. Öğr. Üyesi Uğur CORUH · 16.10.2026

Bugünün planı (3 saat)

Saat	Bölüm	Konu
1	0–3	Temel kavramlar · yönetilen diller · CERT Java · enjeksiyonun kökü
2	4–9	SQL/komut/yol enjeksiyonu · seri durumdan çıkarma · XML/XXE · Python/JS
3	10–16	Bayt kodu · ProGuard/R8 · dize gizleme · Android · SBOM · proje

Kısa tarihçe — yönetilen diller ve kalan açıklar

- 1995 — **Java/JVM**: bellek hatalarının çoğu **dil düzeyinde** biter
- 1998 — SQL enjeksiyonu belgelenir · **2003 OWASP Top 10**
- 2002 — **ProGuard** (sonra **R8**): bayt kod kolay geri çevrildiği için
- 2015 deserialization · **2020–21** SolarWinds ve **Log4Shell** → **SBOM** çağı

Dil **bellek hatasını** çözer; **enjeksiyonu ve bağımlılık riskini** çözmez.

Bu hafta nereye oturuyor?

- **4. hafta:** C/C++ sağlamlaştırma (bellek hataları).
- **Bu hafta (5):** Java/yorumlanan diller — bellek hataları çoğu **çözülmüş**, ama yeni sınıf: **enjeksiyon** ve **bağımlılık**.
- İkili koruma tarafı: **ProGuard/R8** ve **SBOM**.

Öğrenme çıktısı

Bu hafta **ÖÇ.3 / ÖÇ.4** üstünedir.

Sonunda yapabileceğiniz:

- Enjeksiyon sınıfını tanımak ve önlemek (SQL, komut, yol, XML, seri durum)
- ProGuard/R8 ile gizleme ve `-keep` kurallarını yazmak
- Bir **SBOM** üretip bağımlılık riskini yönetmek

Ana fikir

Yönetilen dil bellek hatalarını büyük ölçüde **çözer**; ama **veri ile komutun karışması** (enjeksiyon) her dilde vardır.

Bugün bu ortak kökü ve dile özgü korumaları göreceğiz.

0. Temel kavramlar (sıfırdan)

Yönetilen dil nedir?

- **Yönetilen dil:** belleği **otomatik** yöneten dil (Java, C#, Python, JS).
- Programcı `malloc` / `free` yapmaz; **çöp toplayıcı** (GC) halleder.
- → çoğu bellek hatası (taşma, UAF) **ortadan kalkar**.

JVM ve bayt kodu

- **JVM (Java Virtual Machine):** Java'yı çalıştıran sanal makine.
- **Bayt kodu:** Java kaynağının derlendiği ara biçim (`.class` dosyaları).
- Bayt kodu makineden bağımsız; JVM onu çalıştırır.

Çöp toplayıcı (GC)

- **GC (Garbage Collector):** artık kullanılmayan nesneleri otomatik temizler.
- Programcı belleği elle bırakmaz.
- Bu yüzden UAF/çift bırakma Java'da **yok denecek kadar az**.

Enjeksiyon nedir?

- **Enjeksiyon:** kullanıcı **verisinin** bir **komut** olarak yorumlanması.
- Örnek: kullanıcı adı diye girilen metnin SQL sorgusuna komut olarak sızması.
- Bu haftanın ana teması.

SQL ve veritabanı

- **SQL:** veritabanı sorgulama dili (`SELECT ... WHERE ...`).
- Uygulama, kullanıcı girdisini sorguya koyar.
- Yanlış konursa → **SQL enjeksiyonu**.

Parametrelili sorgu

- **Parametrelili sorgu (prepared statement):** sorgu iskeleti sabit, veriler ayrı **parametre** olarak verilir.
- Veri asla komut olarak yorumlanmaz.
- SQL enjeksiyonuna karşı **asıl** çözüm.

Seri durum (serialization)

- **Serileştirme:** bir nesneyi bayt dizisine çevirme (kaydetmek/göndermek için).
- **Seri durumdan çıkarma (deserialization):** baytları geri nesneye çevirme.
- Güvenilmez baytları çıkarmak **tehlikeli** (kod çalıştırma).

XML ve XXE

- **XML:** yapılandırılmış veri biçimi (etiketlerle).
- **XXE (XML External Entity):** XML'in "dış varlık" özelliğinin kötüye kullanımı → dosya okuma, SSRF.
- XML ayrıştırıcıda dış varlıklar **kapatılır**.

Yol geçişi (path traversal)

- **Yol geçişi:** `../` ile izin verilen klasörün **dışına** çıkma.
- `dosyalar/../../../../etc/passwd` gibi.
- Kanonikleştirme + kök denetimiyle önlenir.

ProGuard ve R8

- **ProGuard / R8:** Java/Android bayt kodunu **küçülten, adları gizleyen** araçlar.
- R8, Android'in varsayılan aracı.
- Ad gizleme + ölü kod eleme + küçültme.

-keep kuralı

- **-keep** : ProGuard/R8'e "bu sınıf/metodun adını **değiştirme**" demek.
- Reflection ile ada göre çağrılan kod korunmalı.
- Yanlış **-keep** → ya çökme ya zayıf gizleme.

Reflection nedir?

- **Reflection:** bir sınıfı/metodu **çalışma anında adıyla** bulup çağırma.
- `Class.forName("...")`, `getMethod("...")`.
- Gizleme bunu bozabilir → `-keep` gerekir.

SBOM nedir?

- **SBOM (Software Bill of Materials):** yazılımın **malzeme listesi** — hangi kütüphaneler, hangi sürümler.
- Bir kütüphanede açık çıkınca "etkilendik mi?" sorusunu **hızlı** yanıtlar.
- Biçimler: CycloneDX, SPDX.

Bağımlılık (dependency)

- **Bağımlılık:** projenizin kullandığı dış kütüphaneler.
- Kendi kodunuz güvenli olsa bile, bir bağımlılıktaki açık sizi etkiler.
- Tedarik zinciri güvenliği.

Şimdi hazırız

Terimler:

yönetilen dil · JVM/bayt kodu · GC · enjeksiyon · SQL/parametrelili sorgu · serileştirme · XML/XXE · yol geçişi · ProGuard/R8 · `-keep` · reflection · SBOM · bağımlılık

Şimdi: yönetilen diller neyi çözer, neyi çözmez?

1. Yönetilen diller ve enjeksiyonun kökü

Yönetilen dil neyi çözer?

- Otomatik bellek yönetimi (GC) → taşma, UAF, çift bırakma **büyük ölçüde yok**.
- Dizi erişimi sınır denetimli → `ArrayIndexOutOfBoundsException` (çökme, sömürü değil).
- Tür güvenliği daha sıkı.

Yönetilen dil neyi ÇÖZMEZ?

- **Enjeksiyon** (veri = komut) — her dilde var.
- **Mantık hataları**, yetki hataları.
- **Bağımlılık** açıkları.
- **Seri durumdan çıkarma** saldırıları.

Bellek hatası azaldı; yeni cephe: enjeksiyon ve tedarik zinciri.

Yönetilen dil: çözer / çözmez — şema

Yönetilen dil neyi çözer, neyi çözmez?

Java/Kotlin/C# çöp toplayıcı + sınır denetimi

ÇÖZER (dil düzeyinde)

tampon taşması
use-after-free
çift serbest bırakma
ilklendirilmemiş değişken
→ bellek güvenliği

ÇÖZMEZ

enjeksiyon (SQL, komut, XML)
güvensiz deserialization
koda gömülü sırlar
zafiyetli bağımlılıklar
mantık/yetki hataları

Dil bir hata SINIFINI kaldırır; kalan sınıflar için hâlâ siz sorumlusunuz.

SEI CERT Oracle Java

- Java için ayrı bir **kural kitabı** (CERT Oracle Coding Standard for Java).
- Aynı yapı: hatalı örnek + uyumlu çözüm + risk.
- Örnek alanlar: enjeksiyon, seri durum, eşzamanlılık, hassas veri.

Java'da hassas veri

- Java `String` **değişmezdir** (immutable) → bellekten silinemez.
- Parola/anahtar için `char[]` kullan, işi bitince **doldur** (`Arrays.fill`).
- GC ne zaman toplayacağını **garanti etmez**.

Enjeksiyonun ortak kökü

Enjeksiyon = **veri** ile **komutun** karışması.

- Kullanıcı verisi bir yorumlayıcıya (SQL, kabuk, XML) **komut** olarak gider.
- Kök aynı; hedef (SQL/kabuk/XML) değişir.

Enjeksiyonun kökü — veri mi, kod mu?

Enjeksiyonun ortak kökü: veri ile komutun karışması

bütün enjeksiyon türleri aynı şemanın farklı yorumlayıcılardaki hâli

HATALI: tek kanal

"...WHERE x=" + girdi

Yorumlayıcı girdiyi
KOMUT olarak ayrıştırır

SQLi · komut enj. · XXE
şablon enjeksiyonu

DOĞRU: ayrı kanal

WHERE x = ? (parametre)

Komut SABİT kalır,
girdi yalnız VERİ

enjeksiyon ailesi
tümüyle kapanır

Kaçış (escaping) son çaredir; asıl çözüm komutu ve veriyi ayrı kanala koymaktır.

Çözümün ortak kökü

Her enjeksiyon türünde aynı ilke:

- **Veriyi koddan ayır** (parametrelili API).
- Veriyi asla **string birleştirmeye** komuta gömme.
- Beyaz liste + bağlama uygun kaçış (son çare).

Enjeksiyon aileleri (bugün)

- SQL enjeksiyonu
- Komut enjeksiyonu
- Yol geçişi
- Seri durumdan çıkarma
- XML/XXE, şablon, XSS

Hepsinin kökü ve çözümü **aynı mantık**.

Bölüm 1 — kısa sinama

1. Yönetilen dil hangi hata sınıfını büyük ölçüde çözer?
2. Neyi çözmez?
3. Enjeksiyonun ortak kökü nedir?

Bölüm 1 — cevaplar

1. **Bellek güvenliği** hatalarını (tampon taşması, use-after-free, çift/sarkan işaretçi) — GC + sınır denetimiyle.
2. **Mantık/enjeksiyon/etkilendirme**, güvensiz deserialization, zayıf kriptoloji, **bağımlılık** açıkları — bunlar dil-üstü, çözülmez.
3. Güvenilmeyen **VERİ** ile **KOD/komutun** aynı kanalda karışması; yorumlayıcı veriyi **komut** olarak ayrıştırır.

2. SQL enjeksiyonu

Hatalı kod

```
String q = "SELECT * FROM kul WHERE ad = '" + ad + "'";  
stmt.executeQuery(q); // HATALI: ad komuta karışır
```

ad içine tırnak ve SQL koyulursa?

Saldırı · örnek

Kullanıcı `ad` olarak şunu girer:

```
' OR '1'='1
```

Sorgu şuna döner:

```
SELECT * FROM kul WHERE ad = '' OR '1'='1'
```

`'1'='1'` her zaman doğru → **tüm satırlar** döner.

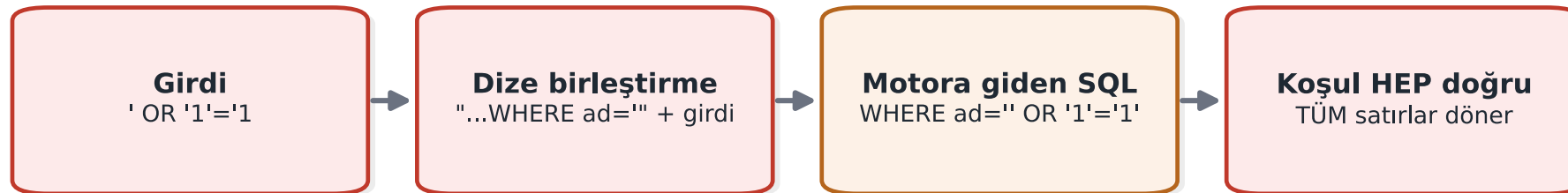
Neden oldu?

- `ad`, sorgu **metnine** birleştirildi.
- Tırnak, veriyi bitirip **komut** kısmına geçmeyi sağladı.
- Veri ile komut **karişti**.

SQL enjeksiyonu adım adım — şema

SQL enjeksiyonu adım adım

girdi tırnağı kapatıp kendi mantığını ekliyor



Parametrelili sorguda motor komutu ÖNCE derler; girdi artık mantık değiştiremez.

Doğru · parametrelili sorgu

```
PreparedStatement ps =  
    con.prepareStatement("SELECT * FROM kul WHERE ad = ?");  
ps.setString(1, ad);    // ad yalnız VERİ  
ps.executeQuery();
```

? bir **parametre**; **ad** asla komut olarak yorumlanmaz.

Neden çalışır?

- Sorgu iskeleti **önceden** veritabanına gönderilir (derlenir).
- `ad` sonradan **veri** olarak bağlanır.
- Tırnak koysa bile veri olarak kalır.

Diğer katmanlar

- **En az yetki:** uygulamanın DB kullanıcısı yalnız gerekeni yapsın.
- **Girdi doğrulama:** yine de beyaz liste.
- **ORM** kullanıyorsanız da parametrelili API'yi kullanın (string birleştirme değil).

Kaçış (escaping) son çaredir

- Elle tırnak kaçırmak **hataya açık**; her DB farklı.
- Öncelik **her zaman** parametrelili sorgu.
- Kaçış yalnız parametrelenemeyen yerlerde, dikkatle.

3. Komut enjeksiyonu

Kabuk var mı yok mu — şema

Komut çalıştırma: kabuk var mı yok mu?

komut enjeksiyonunun tek kaynağı: araya giren kabuk

Kabuklu (tehlikeli)

```
Runtime.exec("sh -c ...")
```

kabuk ; | && > yorumlar
girdi KOMUT olur

ProcessBuilder (güvenli)

```
new ProcessBuilder(  
    "prog", arg1, arg2)
```

argümanlar AYRI geçer
kabuk hiç çalışmaz

Yine de argüman enjeksiyonuna dikkat: '--' ile seçenek/değer sınırını işaretleyin.

Hatalı kod

```
Runtime.getRuntime().exec("ping " + host); // HATALI
```

`host` içine kabuk metakarakteri koyulursa?

Saldırı · örnek

host olarak:

```
example.com; rm -rf veri
```

Kabuk bunu **iki komut** olarak çalıştırır: `ping` **ve** `rm` .

(Örnek sentetik; asla çalıştırmayın.)

Neden oldu?

- Komut bir **kabuğa** metin olarak verildi.
- `;`, `|`, `&&` kabukta **komut ayırıcı**.
- Veri, komut satırına karıştı.

Doğru · argüman dizisi

```
new ProcessBuilder("ping", "-c", "1", host).start();
```

- Kabuk **yok**; `host` tek bir **argüman**.
- Metakarakterler yorumlanmaz.

Ek kurallar

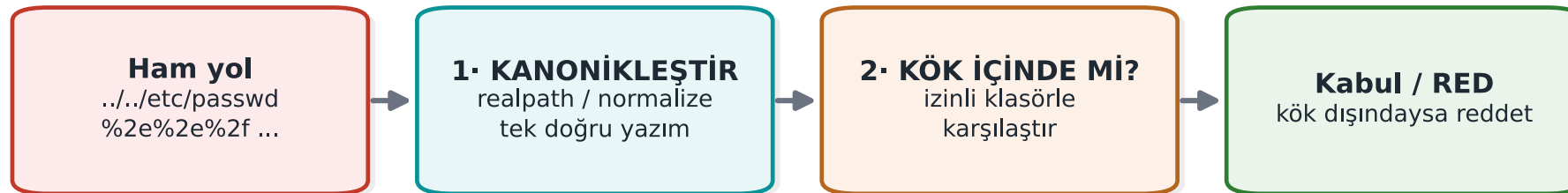
- Mümkünse dış komut **hiç** çalıştırma; kütüphane API'si kullan.
- Çalıştıracaksan: sabit yol, argüman dizisi, beyaz liste.
- ENV33-C benzeri: `system()` /kabuk çağırma.

4. Yol geçişi

Yol geçişini kapatan iki adım — şema

Yol geçişini kapatan iki adım

sıra önemli: önce kanonikleştir, SONRA denetle



Ham dize üzerinde yapılan denetim kodlanmış biçimlerle atlatılır.

Hatalı kod

```
File f = new File("yuklenen/" + adi); // HATALI
```

adi içine ../ koyulursa?

Saldırı · örnek

adi olarak:

```
../../../../etc/passwd
```

Sonuç: `yuklenen/../../../../etc/passwd` → izin verilen klasörün **dışı**.

Doğru · kanonikleştir + kök denetimi

```
Path kok = Paths.get("yuklenen").toRealPath();  
Path hedef = kok.resolve(adi).normalize().toRealPath();  
if (!hedef.startsWith(kok)) throw new SecurityException();
```

Kanonik yol **kökün altında** mı? Değilse reddet.

Yol geçişi · ek

- Dosya adını **beyaz listeye** sınırla (`[a-zA-Z0-9_.-]`).
- Sembolik bağlantılar (`symlink`) kanonikleştirmeyeyle çözülür.
- Kullanıcı adını **doğrudan** dosya adı yapma.

Üç enjeksiyon · ortak ders

Tür	Yanlış	Doğru
SQL	String birleştirme	Parametrelili sorgu
Komut	Kabuğa metin	Argüman dizisi
Yol	../ birleştirme	Kanonikleştir + kök denetimi

Kök aynı: **veriyi koddan/komuttan ayır.**

Bölüm 2–4 — kısa sinama

1. `' OR '1'='1'` neden tüm satırları döndürür?
2. `ProcessBuilder` argüman dizisi neden güvenli?
3. Yol geçişini hangi iki adım kapatır?

Bölüm 2–4 — cevaplar

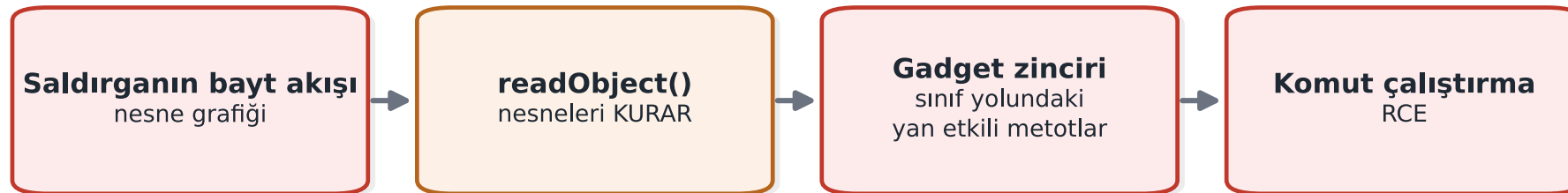
1. Dizeye katılınca `WHERE` koşulunu **her zaman doğru** yapar → tüm satırlar (veri, kod olarak ayrıştırıldı).
2. Argümanlar **kabuğa değil** programa ayrı ayrı geçer; `;` `|` gibi meta-karakter yorumu yok → komut enjeksiyonu kapanır.
3. (1) **Kanonik yola** çevir (`realpath` /normalize), (2) izinli **kök dizin** içinde mi doğrula (allowlist). Kullanıcı girdisini yola doğrudan koyma.

5. Güvensiz seri durumdan çıkarma

Deserialization gadget zinciri — şema

Güvensiz seri durumdan çıkarma

saldırgan kod yazmaz — VAR OLAN sınıfları zincirler



Savunma: `ObjectInputFilter` ile izin listesi ve sonda '!*' (varsayılan-reddet).

Sorun

- Güvenilmez baytları bir nesneye çevirmek **tehlikelidir**.
- Java'da `ObjectInputStream.readObject()` çıkarken **kod** çalıştırabilir (gadget zincirleri).
- Sonuç: uzaktan kod çalıştırma.

Neden bu kadar kötü?

- Çıkarma sırasında nesnelerin **kurucuları/metotları** çalışır.
- Uygun kütüphaneler varsa (gadget), zincirlenip komut çalıştırılabilir.
- Girdi "sadece veri" sanılır ama **kod** gibi davranır.

Doğru · en iyisi: hiç yapma

- Güvenilmez veriyi **native serileştirmeyle** çıkarma.
- Onun yerine **JSON/veri** biçimi + katı şema.
- Yalnız beklediğiniz alanları okuyun.

Zorundaysanız · filtre

```
ObjectInputFilter f = ObjectInputFilter.Config  
    .createFilter("com.uygulama.*;!*"); // yalnız izinli sınıflar  
ois.setObjectInputFilter(f);
```

- **Beyaz liste:** yalnız beklenen sınıflar.
- Derinlik/boyut sınırı da koyun (DoS'a karşı).

6. XML, şablon, XSS

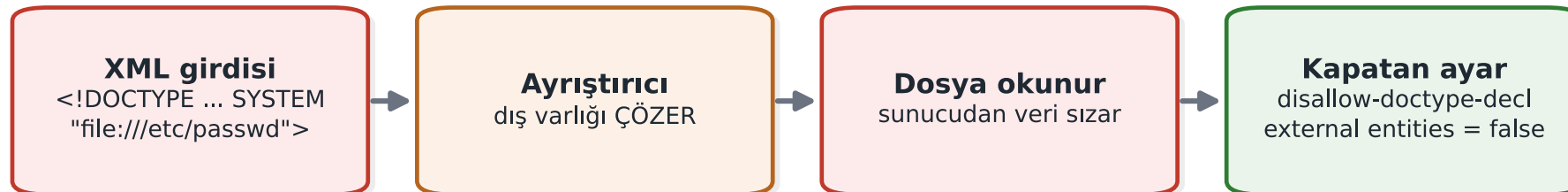
XXE · sorun

- XML'de **dış varlık** tanımlanabilir; ayrıştırıcı onu **çözer**.
- Saldırgan bununla dosya okutabilir ya da sunucudan istek yaptırabilir (SSRF).

XXE — şema

XXE: XML dış varlık saldırısı

ayrıştırıcının 'yardımsever' varsayılını



Kural: XML/YAML/şablon ayrıştırıcılarında gereksiz yetenekleri KAPATIN.

XXE · doğru

```
dbf.setFeature(  
    "http://apache.org/xml/features/disallow-doctype-decl", true);  
// ya da dış varlıkları kapat
```

- **Dış varlıkları kapat**, DOCTYPE'ı reddet.
- Çoğu kütüphanede tek satırlık ayar.

Şablon enjeksiyonu (SSTI)

- Şablon motoruna kullanıcı girdisini **şablon** olarak vermek.
- Motor onu kod gibi çalıştırabilir.
- Kural: kullanıcı girdisi **veri**, şablon **sabit**.

XSS (kısaca)

- **XSS**: kullanıcı verisinin bir web sayfasına **HTML/JS** olarak sızması.
- Çözüm: çıktı **bağlama uygun kodlama** (HTML-encode), CSP.
- Kök yine aynı: veri $\neq$ kod.

7. Python ve JavaScript'te aynı hatalar

Aynı kök, farklı dil

- Enjeksiyon her dilde: Python `os.system` , JS `child_process` .
- Çözüm aynı: argüman dizisi, parametrelili sorgu.

Python · örnekler

- `os.system("cmd " + x)` → komut enjeksiyonu; `subprocess.run([...])` kullan.
- `eval(girdi)` / `pickle.loads(güvenilmez)` → **asla** güvenirilmez veriyle.
- SQL: parametrelili (`cursor.execute("... ?", (x,))`).

ReDoS (düzenli ifade DoS)

- Kötü bir **regex**, belirli girdilerde **üstel** zaman harcayabilir.
- Saldırgan bununla CPU'yu kilitler.
- Çözüm: güvenli regex, girdi uzunluğu sınırı, zaman aşımı.

JavaScript · prototype pollution

- **Prototype pollution:** saldırgan `__proto__` üzerinden tüm nesnelerin **ortak** özelliğini bozar.
- Beklenmedik davranış, yetki atlatma.
- Çözüm: güvenli birleştirme, `Object.create(null)`, kütüphane güncelliği.

Yorumlanan diller · ortak kural

- `eval` , `exec` , `pickle` , dinamik `require` → güvenilmez veriyle **kullanma**.
- Girdi doğrulama + parametrelili API + güncel bağımlılık.

Bölüm 5–9 — kısa sinama

1. Güvensiz deserialization neden kod çalıştırabilir?
2. XXE'yi hangi ayar kapatır?
3. ReDoS nedir, nasıl azaltılır?

Bölüm 5–9 — cevaplar

1. Nesne kurulurken sınıfların yan etkili yöntemleri (`readObject` /**gadget zinciri**) tetiklenir → saldırgan nesne grafiğiyle akış kurar.
2. **Dış varlıkları ve DOCTYPE'ı** kapatan ayrıştırıcı ayarı (`disallow-doctype-decl` , external entities kapalı).
3. **ReDoS**: katastrofik geri izlemeli regex; kötü girdi **üstel** zaman → CPU kilitlenir. Azaltma: doğrusal-zaman motoru (RE2), iç içe niceleyiciden kaçın, uzunluk/zaman sınırı.

8. Bayt kodu ve tersine derleme

Bayt kod vs makine kodu — şema

Bayt kod neden makine kodundan kolay okunur?

bu yüzden Java'da gizleme (ProGuard/R8) daha kritik

Java bayt kodu

sınıf/metot/alan ADLARI durur
tipler durur
yapı korunur

decompiler $\approx$ kaynağı verir

C/C++ makine kodu

adlar atılır
tipler kaybolur
optimizasyon yapıyı bozar

disassembler $\approx$ ham komut

Gizleme hatayı düzeltmez; yalnız 'kolayca okunma' avantajını geri alır.

Java bayt kodu kolay okunur

- `.class` / `.jar` bayt kodu, kaynağa **çok yakın** geri çevrilebilir.
- Araçlar: `javap` (sökme), JD-GUI/CFR/Procyon (tersine derleme).
- Adlar, dizgeler, yapı büyük ölçüde **görünür**.

javap örneği

```
javap -c -p Uygulama.class    # bayt kodunu sök
```

- Metot adları, çağrılar, sabitler görünür.
- Bu yüzden Java'da gizleme **daha da** gerekli.

Neden C'den daha kolay?

- Bayt kodu **yüksek seviyeli** ve tür bilgisi taşır.
- Değişken/metot adları çoğunlukla korunur.
- → tersine mühendislik ucuz; gizleme şart.

9. ProGuard ve R8

ProGuard/R8 ne yapar?

Üç iş:

- **Küçültme (shrink):** kullanılmayan kodu atar.
- **İyileştirme (optimize):** kodu sadeleştirir.
- **Gizleme (obfuscate):** adları anlamsızlaştırır (`a` , `b` , `c`).

Ad gizleme örneği

```
LisansDenetleyici.dogrula() → a.b()
```

- Anlamlı adlar kaybolur.
- Tersine mühendis için harita zorlaşır.
- Ölü kod atılınca ikili küçülür.

-keep neden gerekir?

- Reflection ile **adıyla** çağrılan kod (API, giriş noktaları, JNI) korunmalı.
- Gizleme adı değiştirirse çağrı **kırılır**.

```
-keep class com.uygulama.Api { public *; }
```

-keep dengesi — şema

-keep dengesi

doğrusu: yalnız dışarıdan erişilen giriş noktalarını keep et

ÇOK keep

reflection, serileştirme,
tüm public API korunur

- adlar değişmez
- gizleme ANLAMSIZ

AZ keep

yalnız dış giriş noktaları

- adlar kısalır
- ama reflection KIRILIR
(çalışma anı hatası)

Her -keep kuralı bir gizleme boşluğudur; en dar kuralı yazın ve test edin.

-keep dengesi

- Çok az **-keep** : uygulama çöker (reflection kırılır).
- Çok fazla **-keep** : gizleme zayıflar (her şey açık kalır).
- Yalnız **gerçekten gereken** giriş noktalarını tut.

Eşleme dosyası (mapping)

- R8 bir `mapping.txt` üretir: gizli ad → gerçek ad.
- **Gizli tutulur** (dağıtılmaz), ama çökme izlerini çözmek için saklanır.
- Sürüm kimliğiyle birlikte arşivlenir.

10. Dize gizleme ve reflection

Java'da dizgeler açık

- `.class` içindeki sabit dizgeler **düz** görünür (`strings` , tersine derleyici).
- Hassas URL, anahtar parçası, mesajlar sızar.

Dize gizleme

- Dizgeler derlemede **kodlanır**, kullanımda çözülür.
- iv. haftadaki kuralla aynı: `strings` 'i durdurur, **anahtar saklamaz**.
- Çözme kodu da bytecode'da → sınırlı koruma.

Reflection ile gizleme çatışması

- Ada göre çağrı (`getMethod("dogru1a")`) gizlemeden **sonra** kırılır.
- Çözüm: ya `-keep` ya reflection'ı bırakıp doğrudan çağrı.
- Reflection'ı azaltmak hem güvenlik hem gizleme için iyidir.

11. Android R8 ve ölçme

Android'de R8

- R8, Android derlemesinin **varsayılan** küçültücü/gizleyicisi.
- `minifyEnabled true` + `proguard-rules.pro` .
- DEX (Android bayt kodu) üretir.

Gizlemenin etkisini ölçmek

- Gizleme öncesi/sonrası:
 - anlamlı ad oranı
 - düz metin hassas dize sayısı
 - APK boyutu
- Ölç ve S9'a yaz (9. haftadaki ölçme mantığı).

Java için statik analiz

- SpotBugs, Error Prone, SonarQube.
- CERT Java kurallarına yakın denetimler.
- CI'da her birleştirmede.

Bölüm 8–13 — kısa sinama

1. Java bayt kodu neden C'den kolay okunur?
2. `-keep` dengesi nedir?
3. Reflection gizlemeyle neden çatışır?

Bölüm 8–13 — cevaplar

1. Bayt kod **yüksek seviyeli**; tip ve **ad meta verisi** (sınıf/alan/metot) korunur → decompiler neredeyse kaynağı verir. C, adları atarak makine koduna derlenir.
2. **-keep dengesi**: çok keep → gizleme zayıflar; az keep → reflection/serileştirme kırılır. Yalnız **dış giriş noktalarını** keep et.
3. Reflection adları **dizayle** çağırır; gizleyici adı değiştirince dize eşleşmez → çalışma anında hata. O adları keep etmek gerekir (gizleme boşluğu).

12. Bağımlılık güvenliği ve SBOM

Sorun · tedarik zinciri

- Kendi kodunuz kusursuz olsa bile, kullandığınız **kütüphanelerdeki** açık sizi etkiler.
- Modern uygulama yüzlerce bağımlılık içerir.
- "Hangi sürüm bende var?" sorusu kritik.

Gerçek örnekler

- Log4Shell (Log4j): tek bir kütüphane açığı, milyonlarca sistemi etkiledi.
- Kötü niyetli paket (typosquatting): benzer adlı sahte paket.
- Bakımsız/terk edilmiş bağımlılık.

SBOM nedir? (hatırlatma)

- Yazılımın **malzeme listesi**: hangi bileşen, hangi sürüm, hangi lisans.
- Bir açık duyurulunca "etkilendim mi?" **hızlı** yanıtlanır.

SBOM biçimleri

- **CycloneDX** (OWASP) — güvenlik odaklı, yaygın.
- **SPDX** (Linux Foundation) — lisans odaklı da güçlü.
- Araçlarla otomatik üretilir (derleme adımımda).

VEX nedir?

- **VEX (Vulnerability Exploitability eXchange):** "bu açık bende var ama **sömürülebilir değil**" bilgisini paylaşır.
- Gürültüyü azaltır: her CVE panik değil.
- SBOM'a eşlik eder.

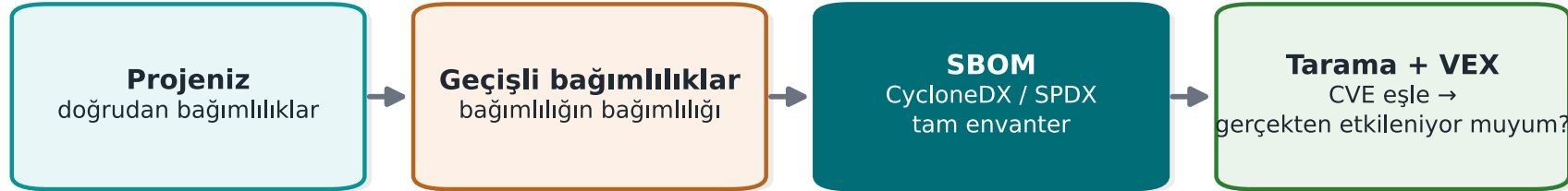
Bağımlılık yönetimi kuralları

- Sürümleri **sabitle** (pin); rastgele "en son" çekme.
- Bilinen açıkları tara (`dependency-check` , `npm audit` , `pip-audit`).
- Güvenli güncelleme süreci; kritik yamaları hızlı uygula.

SBOM üretimi · örnek akış

Tedarik zinciri: SBOM neden gerekli?

Log4Shell'de asıl sorun 'bende var mı?' sorusuydu



Sürümleri sabitleyin: yeniden üretilebilir derleme + beklenmedik güncellemeye karşı koruma.

Her sürümün SBOM'u sürüm kimliğiyle saklanır.

Projeye bağ

- **S13:** güvenli geliştirme süreci + SBOM.
- Her sürüm için güncel SBOM ve bağımlılık taraması.
- Devredilen gereksinimler: "güvenli güncelleme" üst uygulamaya (13. hafta).

Bölüm 14 — kısa sinama

1. SBOM ne işe yarar?
2. VEX neden gürültüyü azaltır?
3. Bağımlılık sürümlerini neden sabitleyiz?

Bölüm 14 — cevaplar

1. **SBOM** = bileşen/bağımlılık envanteri; yeni açık çıkınca "bende var mı?" sorusunu **hızla** yanıtlar.
2. **VEX** açığın üründe gerçekten **sömürülebilir** olup olmadığını bildirir; etkilenmeyenler "not affected" → yanlış alarm azalır.
3. Sabit sürüm = **yeniden üretilebilir** derleme + tedarik zinciri güvenliği; beklenmedik/zararlı güncellemeyi engeller.

Proje ve kapanış

Proje · S9/S13 (Java tarafı)

- [] Enjeksiyona açık noktaları parametrelili API'ye çevir (SQL, komut, yol).
- [] Güvensiz deserialization varsa filtre/şema.
- [] R8/ProGuard ile gizleme; `-keep` yalnız gereken giriş noktaları.
- [] Dize gizleme; hassas dize `strings` /tersine derlemede yok.
- [] SBOM üret + bağımlılık taraması (S13).

Çözümlü kendini sinama

Soru 1

Yönetilen dil hangi hata sınıfını çözer, neyi çözmez?

Cevap: Bellek hatalarını (taşma, UAF) büyük ölçüde çözer. Enjeksiyon, mantık/yetki hataları, bağımlılık ve deserialization saldırılarını **çözmez**.

Soru 2

' OR '1'='1' neden tüm satırları döndürür?

Cevap: Girdi string birleştirmeyele sorguya girer; tırnak veriyi bitirir, OR '1'='1' her zaman doğru olur. Parametrelili sorgu bunu engeller.

Soru 3

Komut enjeksiyonunu `ProcessBuilder` nasıl önler?

Cevap: Kabuk kullanmadan argümanları **dizi** olarak verir; metakarakterler (`;`, `|`) yorumlanmaz, girdi tek argüman kalır.

Soru 4

Yol geçişini hangi iki adım kapatır?

Cevap: Kanonikleştirme (`normalize` / `toRealPath`) + kök denetimi (`startsWith(kok)`). Ayrıca dosya adı beyaz listesi.

Soru 5

Güvensiz deserialization neden tehlikelidir? Çözüm?

Cevap: Çıkarma sırasında nesne metotları çalışır; gadget zinciriyle kod çalıştırılabilir. Çözüm: native serileştirme yerine veri biçimi + şema; zorunda kalınırsa `ObjectInputFilter` beyaz listesi.

Soru 6

XXE nedir, hangi ayar kapatır?

Cevap: XML dış varlıklarının kötüye kullanımı (dosya okuma/SSRF). Dış varlıkları ve DOCTYPE'ı kapatan ayrıştırıcı ayarı kapatır.

Soru 7

ReDoS ve prototype pollution nedir?

Cevap: ReDoS: kötü regex'in üstel zaman harcaması (DoS). Prototype pollution: JS'te `__proto__` üzerinden tüm nesneleri bozma. İkisi de girdi denetimi + güvenli API ister.

Soru 8

Java bayt kodu neden gizlemeye daha çok ihtiyaç duyar?

Cevap: Bayt kodu yüksek seviyeli, tür ve ad bilgisi taşır; kaynağa çok yakın tersine derlenir. Adlar/dizgeler açık görünür.

Soru 9

-keep dengesi nedir?

Cevap: Çok az **-keep** → reflection kırılır, uygulama çöker. Çok fazla → gizleme zayıflar. Yalnız gerçekten reflection'la çağrılan giriş noktaları tutulur.

Soru 10

Dize gizleme Java'da anahtar saklar mı?

Cevap: Hayır. `strings` /statik taramayı durdurur ama çözülen dize bellekte açıktır ve çözme kodu bytecode'dadır. Anahtar için farklı koruma gerekir.

Soru 11

SBOM ve VEX ne işe yarar?

Cevap: SBOM bileşen listesidir; açık duyurulunca etkilenip etkilenmediğinizi hızla söyler. VEX, açığın sömürülebilir olup olmadığını belirtir, gürültüyü azaltır.

Soru 12

Tüm enjeksiyon türlerinin ortak kökü ve çözümü?

Cevap: Kök: veri ile komutun karışması. Çözüm: veriyi koddan **ayırmak** (parametrelili/argüman dizisi API), string birleştirmeyele komut kurmamak.

Özet: bu haftanın tek cümlesi

Yönetilen dil bellek hatalarını çözer ama **enjeksiyon her dilde vardır**; çözüm her zaman veriyi komuttan ayırmak, koruma tarafında ise gizleme (R8) ve **bağımlılık/SBOM** yönetimidir.

Gelecek hafta

6. hafta — RASP ve çalışma anı korumaları

Kurcalama/hata ayıklama tespiti, bütünlük denetimi, tepki politikası ve katmanlı savunma.

Ek A · Bir enjeksiyonu baştan sona

Bağlam · giriş formu

Kullanıcı adı ve parola ile giriş yapan bir uygulama.

```
String q = "SELECT * FROM kul WHERE ad='" + ad +  
            "' AND parola='" + p + "'";
```

Nerede yanlış?

Adım 1 · saldırgan girdisi

ad alanına:

```
yonetici' --
```

-- SQL'de **yorum** başlatır; sonrası yok sayılır.

Adım 2 · sorgu ne oluyor?

```
SELECT * FROM kul WHERE ad='yonetici' -- ' AND parola='...'
```

Parola denetimi **yorumda** kaldı → parolasız **yonetici** girişi.

Adım 3 · etki

- Kimlik doğrulama **atlatıldı**.
- Daha ileri: `UNION SELECT` ile başka tabloları okuma.
- `; DROP TABLE` (yığın sorgu destekliyse) veri kaybı.

Adım 4 · düzeltme

```
PreparedStatement ps = con.prepareStatement(  
    "SELECT * FROM kul WHERE ad=? AND parola_ozet=?");  
ps.setString(1, ad);  
ps.setString(2, ozetle(p));
```

ad ve parola artık **veri**; **--** etkisiz.

Adım 5 · katmanlı savunma

- **Parolayı düz saklama:** özet + tuz (3. hafta).
- **En az yetki:** DB kullanıcısı yalnız `SELECT` .
- **Girdi doğrulama:** kullanıcı adı beyaz liste.
- **Günlük/izleme:** başarısız giriş denemeleri.

Ders

- Tek kök: veri komuta karıştı.
- Asıl çözüm: parametrelili sorgu.
- Ama derinlemesine savunma her katmanda.

Ek B · Deserialization mini vaka

Kurgu

Bir uygulama, kullanıcının gönderdiği "oturum nesnesini" native serileştirmeyeyle çıkarıyor:

```
Object o = new ObjectInputStream(giris).readObject(); // TEHLİKELİ
```

Ne olabilir?

- Saldırgan, sınıf yolunda bulunan kütüphanelerden bir **gadget zinciri** hazırlar.
- Çıkarma sırasında bu zincir çalışır → komut çalıştırma.
- Girdi "veri" sanıldı, ama kod gibi davrandı.

Düzeltilme 1 · biçim değiştir

- Native serileştirme yerine **JSON** + katı şema.
- Yalnız beklenen alanlar okunur.
- Nesne grafiği "canlanmaz".

Düzeltilme 2 · zorundaysan filtre

```
ois.setObjectInputFilter(ObjectInputFilter.Config  
    .createFilter("com.uygulama.Oturum;!*")); // yalnız bu sınıf
```

- Beyaz liste + derinlik/boyut sınırı.

Vaka · ders

- Güvenilmez veriyi "canlı nesneye" çevirmek risklidir.
- En güvenlisi: hiç yapmamak (veri biçimi kullan).
- Zorunda kalınca: katı beyaz liste.

Ek C · Dil dil enjeksiyon

Aynı hata, üç dil

Tehlike	Java	Python	JS
Komut	<code>Runtime.exec(str)</code>	<code>os.system(str)</code>	<code>exec(str)</code>
Güvenli	<code>ProcessBuilder(...)</code>	<code>subprocess.run(...)</code>	<code>execFile(...)</code>

Aynı hata, veri çıkarma

Tehlike	Java	Python	JS
Kod çalıştırma	<code>readObject</code>	<code>pickle.loads</code> , <code>eval</code>	<code>eval</code> , <code>Function</code>
Güvenli	JSON + şema	<code>json.loads</code>	<code>JSON.parse</code>

Kök hep aynı: veri $\neq$ kod.

Üç dil · tek kural

- Dinamik "çalıştır" (`eval` , `exec` , `pickle` , `readObject`) → güvenilmez veriyle **asla**.
- Komut → argüman dizisi.
- SQL → parametrelili.
- Girdi → beyaz liste + reddetme.

Ek A–C · özet

- Enjeksiyon dile bağlı değil; **kalıba** bağlı.
- Kalıbı tanı: "kullanıcı verisi bir yorumlayıcıya komut olarak mı gidiyor?"
- Tanıdıysan çözüm hazır: ayır.

Ek D · R8 gizleme adım adım

Adım 1 · aç

build.gradle :

```
buildTypes { release {  
    minifyEnabled true  
    proguardFiles(..., 'proguard-rules.pro')  
}}
```

Sürüm derlemesinde küçültme + gizleme açılır.

Adım 2 · giriş noktalarını koru

```
-keep class com.uygulama.PublicApi { public *; }  
-keepclassmembers class * {  
    @com.uygulama.Reflected *;  
}
```

Yalnız reflection/JNI ile çağrılanları tut.

Adım 3 · derle ve doğrula

- APK'yı tersine derle; adların `a`, `b` olduğunu gör.
- `mapping.txt` 'i **sakla** (dağıtma).
- Reflection kullanan akışları **test et** (çökme yok mu?).

Adım 4 · ölç

- Anlamlı ad oranı: önce %90 → sonra %10.
- Düz hassas dize sayısı: önce N → sonra 0.
- APK boyutu: küçüldü mü?

Ölçümü S9'a yaz.

Adım 5 · dize gizleme ekle

- Hassas dizgeleri kodla, kullanımda çöz, hemen sil.
- `strings` /tersine derlemede görünmemeli.
- Anahtar değil; yalnız statik tarama önlemi.

Ek E · SBOM adım adım

Adım 1 · üret

```
# örnek: CycloneDX eklentisi  
mvn org.cyclonedx:cyclonedx-maven-plugin:makeBom
```

Derlemede bir `bom.json` / `bom.xml` üretilir.

Adım 2 · tara

```
dependency-check --scan . --format HTML
```

Bilinen açıkları (CVE) bağımlılıklara eşler.

Adım 3 · değerlendir (VEX)

- Her açık için: bende **var mı? sömürülebilir mi?**
- Sömürülemezse VEX ile işaretle (gürültü azalır).
- Sömürülebilirse: güncelle ya da azalt.

Adım 4 · arşivle

- SBOM + tarama raporu, **sürüm kimliğiyle** saklanır.
- Yeni açık çıkınca: "hangi sürümde bu bağımlılık vardı?" hızlı yanıt.

Adım 5 · süreç

- Sürümleri sabitle; otomatik "en son" çekme.
- Kritik yamaları hızlı uygula.
- Güvenli güncellemeyi karşılayamıyorsan → **devret** (13. hafta).

Ek F · Hızlı başvuru

Sık yapılan hatalar

- String birleştirmeye SQL/komut kurmak.
- Güvenilmez veriyle `eval` / `pickle` / `readObject` .
- XML'de dış varlıkları açık bırakmak.
- `-keep *` ile her şeyi korumak (gizleme boşa).
- Bağımlılıkları taramamak.

Sözlük

Terim	Anlam
Enjeksiyon	Veri = komut karışması
Parametrelili sorgu	Veri ayrı bağlanır
Deserialization	Bayt → nesne (riskli)
XXE	XML dış varlık kötüye kullanımı
R8/ProGuard	Küçültme + ad gizleme
SBOM/VEX	Bileşen listesi / sömürülebilirlik

Kontrol listesi

- [] SQL parametrelili, komut argüman dizisi
- [] Yol kanonik + kök denetimi
- [] Deserialization filtreli/şemalı
- [] XXE kapalı
- [] R8 açık, `-keep` minimal, ölçüldü
- [] SBOM üretildi, bağımlılık taraması temiz

Son söz (5. hafta)

Dil bellek hatasını çözse de **enjeksiyon ve tedarik zinciri** sizin sorumluluğunuzdadır.

Veriyi koddan ayır; ikiliyi R8 ile gizle; bağımlılığı SBOM ile yönet.